TP1 - Sécurité et Confidentialité

Mathieu WAHARTE - APP5

Partie 1 : Algorithmes de hachage

On peut obtenir la liste des algorithmes supportés par openssl avec la commande `openssl dgst -list` :

```
bluefir  TP1  main  openssl dgst -list
Supported digests:
-blake2b512                    -blake2s256                    -md4
-md5                           -md5-sha1                      -ripemd
-ripemd160                     -rmd160                        -sha1
-sha224                        -sha256                        -sha3-224
-sha3-256                      -sha3-384                      -sha3-512
-sha384                        -sha512                        -sha512-224
-sha512-256                    -shake128                      -shake256
-sm3                           -ssl3-md5                      -ssl3-sha1
-whirlpool
```

1. On va comparer les temps de calcul des fonctions de hachage blake2b512 et SHA3-256 sur le fichier `/boot/vmlinuz` en utilisant la commande `time openssl dgst [-blake2b512 | -sha3-256] /boot/vmlinuz` :
   Le temps pour blake2b512 est :

   ```
   root@user-VMware-Virtual-Platform:/home/user/Desktop# time openssl dgst -blake2b512 /boot/vmlinuz
   BLAKE2B-512(/boot/vmlinuz)= be6338669b9f0c237ccd0b6494239a13570722ae5ea8196eba6db485b9735faf827a71903bd9364af29c62216bd2299429a26f2a2fa81aabbf19a0d2dbb8b0e5

   real    0m0.043s
   user    0m0.035s
   sys     0m0.008s
   ```

   Le temps pour sha3-256 est :

   ```
   root@user-VMware-Virtual-Platform:/home/user/Desktop# time openssl dgst -sha3-512 /boot/vmlinuz
   SHA3-512(/boot/vmlinuz)= 2d0039dabd15d445fd0b0308bfc06e55489bf5dca0041a7116e54d0cb1e526bce0062d0621dd527d172e29f4311212aeddfc84fa924ff3ed00fcc8dbf9e273ad

   real    0m0.107s
   user    0m0.101s
   sys     0m0.007s
   ```

   Le plus rapide est blake2b512 avec un temps de 0.043s contre 0.107s pour sha3-256. L'ordre de grandeur est de la dizaine de millisecondes pour blake2b512 et de la centaine de millisecondes pour sha3-256.

2. Nous allons vérifier la détection du changement du fichier source. Pour cela, nous allons copier le fichier `/boot/vmlinuz` dans un nouveau fichier `modif`, puis ajouter une ligne à la fin de ce fichier. Ensuite, nous allons recalculer le hash du fichier modifié et le comparer avec le hash du fichier original en utilisant le même algorithme.
   Voici les commandes utilisées :

```
cp /boot/vmlinuz modif
echo 1>>modif
openssl dgst -blake2b512 /boot/vmlinuz > original_hash
openssl dgst -blake2b512 modif > modified_hash
diff original_hash modified_hash
```

Après avoir exécuté ces commandes, nous constatons que les deux hash sont différents (je les ai exécutés sur un autre fichier car WSL n'a pas /boot/vmlinuz).

```
bluefir  src  main  cp /usr/bin//zstd modif
bluefir  src  main  ls
binary_hash1  f1    f2    f.pem  passwd.c    private_f_des3.pem  private_key.pem  secret1
binary_hash2  f1.c  f2.c  modif  passwd.sig  private_f.pem       public_f.pem     secret2
bluefir  src  main  echo 1>>modif
bluefir  src  main  cp /usr/bin//zstd modif
bluefir  src  main  openssl dgst -blake2b512 modif > original_hash
bluefir  src  main  echo 1>>modif
bluefir  src  main  openssl dgst -blake2b512 modif > modified_hash
bluefir  src  main  diff original_hash modified_hash
1c1
< BLAKE2B-512(modif)= ca3ab9c3533843bad1443f1e35d15d64666ab3d5c96737f54874fc2ce9c1c3467ae7ea00dfa4edbff93ce631c803eb5a6d9aa19399be0767b7376726c85422ff
---
> BLAKE2B-512(modif)= 26930f30218e94536876c53b006efce1700214fb4b36b567c2e47b615c7f90691b2cf5ab2a05c4a1b777683cb855c0cbcf4fbec090dfd2a4fc805f1cc1c113fa
bluefir  src  main
```

On peut conclure que même un petit changement dans le fichier source (comme l'ajout d'une ligne) entraîne un changement significatif dans le hash calculé. Cela démontre l'efficacité de la fonction de hachage pour détecter les modifications dans les données.

3. Vérifions les deux représentations hexadécimales.
   Leur taille est bien de 128 octets :

```
bluefir  TP1  main  ls -l binary_hash1
-rw-r--r-- 1 bluefir bluefir 128 Jan 15 14:39 binary_hash1
```

   Et les deux représentations sont bien différentes :

```
bluefir  TP1  main  cmp binary_hash1 binary_hash2
binary_hash1 binary_hash2 differ: byte 20, line 1
```

   Le rang de la première différence est 20.

4. Les deux blocs sont différents mais leurs empreintes MD5 sont identiques. Cela signifie qu'il y a eu une collision de la fonction de hachage MD5.

```
bluefir  TP1  main  md5sum binary_hash1
79054025255fb1a26e4bc422aef54eb4  binary_hash1
bluefir  TP1  main  md5sum binary_hash2
79054025255fb1a26e4bc422aef54eb4  binary_hash2
```

   La valeur de collision est : `79054025255fb1a26e4bc422aef54eb4`.

5. On constate que l'on peut générer de nouvelles collisions en concaténant un même suffixe aux deux blocs collisionnaires :

   - Commande 1 : `cat binary_hash1 binary_hash1 | md5sum`

- Commande 2 : `cat binary_hash2 binary_hash1 | md5sum`

```
bluefir  TP1  main  cat binary_hash2 binary_hash1 | md5sum
84bb273813c848889b07d371222710b2  -
bluefir  TP1  main  cat binary_hash1 binary_hash1 | md5sum
84bb273813c848889b07d371222710b2  -
```

En changeant le suffixe :

- Commande 1 : `cat binary_hash1 binary_hash2 | md5sum`
- Commande 2 : `cat binary_hash2 binary_hash2 | md5sum`

```
bluefir  TP1  main  cat binary_hash1 binary_hash2 | md5sum
ca2ec36baddf27a5b6567d144af79086  -
bluefir  TP1  main  cat binary_hash2 binary_hash2 | md5sum
ca2ec36baddf27a5b6567d144af79086  -
```

Les résultats des 2 commandes sont identiques par suffixe. On a donc bien généré de nouvelles collisions en ajoutant un suffixe commun aux deux blocs initiaux.

En cherchant, on trouve que MD5 est une fonction itérative qui maintient un état interne mis à jour bloc par bloc. Si deux blocs distincts (ici `binary_hash1` et `binary_hash2`) laissent l'algorithme dans le même état interne après leur traitement (c'est la collision observée), alors tout suffixe commun ajouté ensuite sera traité à partir du même état et produira le même haché final. Ce comportement n'est pas dû à une propriété commutative (l'ordre des blocs compte en général), mais au fait que l'état interne est identique avant le traitement du suffixe.

Partie 2 : Chiffrement symétrique

On peut obtenir la liste des algorithmes de chiffrement symétrique supportés par openssl avec la commande `openssl enc -list` :

```
bluefir  TP1  main  openssl enc -list
Supported ciphers:
-aes-128-cbc              -aes-128-cfb              -aes-128-cfb1
-aes-128-cfb8             -aes-128-ctr              -aes-128-ecb
-aes-128-ofb              -aes-192-cbc              -aes-192-cfb
-aes-192-cfb1             -aes-192-cfb8             -aes-192-ctr
-aes-192-ecb              -aes-192-ofb              -aes-256-cbc
-aes-256-cfb              -aes-256-cfb1             -aes-256-cfb8
-aes-256-ctr              -aes-256-ecb              -aes-256-ofb
-aes128                   -aes128-wrap              -aes192
-aes192-wrap              -aes256                   -aes256-wrap
-aria-128-cbc             -aria-128-cfb             -aria-128-cfb1
-aria-128-cfb8            -aria-128-ctr             -aria-128-ecb
-aria-128-ofb             -aria-192-cbc             -aria-192-cfb
-aria-192-cfb1            -aria-192-cfb8            -aria-192-ctr
```

6. En faisant `openssl enc -aes-256-cbc -salt -iter 10 -p -in /etc/passwd -out passwd.c` pour `password123` on obtient successivement :

```
bluefir  TP1  main  openssl enc -aes-256-cbc -salt -iter 10 -p -in /etc/passwd -out passwd.c
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
salt=AA8EDE45075190E7
key=69FFEC34882DD9C962A0313EA18B3E0F7A8585BA46A5B245E14E64BCF9B9A728
iv =9B4D5F633F65C6F6759E2E0AEAA590A9
```

et

```
bluefir  TP1  main  openssl enc -aes-256-cbc -salt -iter 10 -p -in /etc/passwd -out passwd.c
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
salt=0101A414957858FB
key=5C0B5A78E9A4950C053D883DA0C6A078A56ACCFA3C98F78A0381E5117CE2321C
iv =351B839AD9F30ACDD89E77C804E3C547
```

On remarque que les valeurs de salt et key sont différentes. En effet, la valeur de salt est générée aléatoirement à chaque exécution de la commande. La clé est dérivée du mot de passe et du salt, donc elle change aussi.

7. Comme on s'en doutait, en utilisant `-nosalt` on obtient la même valeur de salt et key à chaque exécution de la commande avec le même mot de passe (ici `password123`) :

```
bluefir  TP1  main  openssl enc -aes-256-cbc -nosalt -iter 10 -p -in /etc/passwd -out passwd.c
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
key=1DD6425F93DD1A2FB410E7AE98222C1EB371D986A61FC065F065ACF104F25442
iv =EC1BDA6CB50CB74BBAC76C4F9C27ED9E
bluefir  TP1  main  openssl enc -aes-256-cbc -nosalt -iter 10 -p -in /etc/passwd -out passwd.c
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
key=1DD6425F93DD1A2FB410E7AE98222C1EB371D986A61FC065F065ACF104F25442
iv =EC1BDA6CB50CB74BBAC76C4F9C27ED9E
```

8. On utilise `openssl enc -des-ecb -nosalt -provider legacy -provider default -iter 10 -p -in f1 -out f1.c` pour chiffrer le fichier f1 avec le mot de passe `mdp` et de même pour f2 :

```
bluefir  TP1  main  openssl enc -des-ecb -nosalt -provider legacy -provider default -iter 10 -p -in f1 -out f1.c
enter DES-ECB encryption password:
Verifying - enter DES-ECB encryption password:
key=E76E1FF6ED7A3390
```

```
bluefir  TP1  main  openssl enc -des-ecb -nosalt -provider legacy -provider default -iter 10 -p -in f2 -out f2.c
enter DES-ECB encryption password:
Verifying - enter DES-ECB encryption password:
key=E76E1FF6ED7A3390
```

On obtient la même clé pour les deux.

Maintenant comparons les fichiers chiffrés en hexadécimal :

```
TP1 > f2.c
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   Decoded Text
52 B0 64 49 0C 42 C6 FA F1 F6 F7 49 78 1C 4E 4B   R . d I . B . . . . . I x . N K
CB 6B C8 95 D0 AA 36 3E +                         . k . . . . 6 > +
```

```
TP1 > f1.c
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   Decoded Text
F1 F6 F7 49 78 1C 4E 4B 52 B0 64 49 0C 42 C6 FA   . . . I x . N K R . d I . B . .
CB 6B C8 95 D0 AA 36 3E +                         . k . . . . 6 > +
```

On remarque une permutation des blocs entre les deux fichiers chiffrés (les blocs en roses, vert et bleus sont identiques). En effet, le mode ECB chiffre chaque bloc indépendamment, donc si deux blocs en clair sont identiques, ils seront chiffrés de la même manière. Comme les fichiers f1 et f2 contiennent les mêmes blocs mais dans un ordre différent, les blocs chiffrés apparaissent dans un ordre différent dans les fichiers chiffrés.
Cela peut révéler des motifs dans les données chiffrées, ce qui est une faiblesse de ce mode de chiffrement.

9. Une explication pour le choix des options de chiffrement est la sécurité. Le mode ECB n'est pas recommandé pour chiffrer des données sensibles car il ne fournit pas une sécurité suffisante contre certaines attaques. Il existe d'autres modes de chiffrement plus sûrs, comme le mode CBC (Cipher Block Chaining) ou GCM (Galois/Counter Mode), qui introduisent de l'aléatoire et de la dépendance entre les blocs pour renforcer la sécurité.
   L'utilisation du salage et d'un nombre d'itérations permettent d'augmenter la sécurité du chiffrement en rendant plus difficile les attaques par force brute ou par dictionnaire. Le salage ajoute une valeur aléatoire au mot de passe avant le dériver la clé de chiffrement, ce qui empêche l'utilisation de tables arc-en-ciel pour casser les mots de passe. Le nombre d'itérations augmente le temps nécessaire pour dériver la clé à partir du mot de passe, ce qui rend les attaques par force brute plus coûteuses en termes de temps et de ressources.

Partie 3 : Génération de clé RSA

10. On peut générer une paire de clé RSA avec la commande `openssl genrsa -out private_f.pem 1024` puis extraire la clé publique avec `openssl rsa -in private_f.pem -pubout -out public_f.pem` avant de l'afficher (avec `openssl rsa -in private_f.pem -text -noout`) :

```
bluefir  TP1  main  openssl genrsa -out private_f.pem 1024
bluefir  TP1  main  openssl rsa -in private_f.pem -pubout -out public_f.pem
writing RSA key
bluefir  TP1  main  openssl rsa -in private_f.pem -text -noout
Private-Key: (1024 bit, 2 primes)
modulus:
    00:e2:8c:21:23:11:5c:94:bd:9a:8f:6b:39:67:81:
    68:16:72:5a:eb:47:bd:96:43:5a:a6:2f:3e:a5:d6:
    13:77:5a:bd:af:f8:b3:45:6c:71:22:a1:a0:6a:42:
    ed:2d:f8:b2:58:4b:ce:ce:ea:84:de:1b:40:7f:a3:
    02:bd:e2:57:17:77:98:52:05:1c:87:8b:c7:d9:2b:
    fc:e1:52:ca:ce:73:06:0d:ce:e5:25:cb:9a:b6:ff:
    f2:c7:a0:75:8c:5a:59:ab:bf:4a:ab:f8:cd:36:7f:
    cf:b7:c4:e8:a2:af:7e:58:51:42:23:10:68:30:ea:
    76:bd:ab:1d:77:3b:04:7d:c5
publicExponent: 65537 (0x10001)
privateExponent:
    72:77:cf:7e:82:95:3e:ea:96:a0:04:d3:9a:82:fe:
    6e:e9:15:1b:70:c2:50:9c:73:bf:28:7c:05:1b:27:
    f6:d4:b8:a7:a3:88:21:79:eb:a8:4d:f6:59:fa:bb:
    10:d1:6a:7b:c6:c2:16:36:d5:2f:cc:a1:48:5c:bb:
    53:ea:7b:8a:02:9a:8c:76:bc:8f:96:c9:82:8a:ee:
    89:d4:fa:d0:a4:02:8b:05:27:d0:74:a3:9b:99:e9:
    92:a6:b7:8a:de:bf:d0:95:1d:52:6b:09:69:4e:bf:
    76:19:e2:87:a7:6b:fc:5d:44:4d:f6:3f:ad:f9:0d:
    db:85:7a:05:a3:31:9c:79
prime1:
    00:f2:6b:39:07:a9:57:38:07:a2:13:22:0e:71:e9:
    82:f6:16:84:34:f4:eb:8b:d9:8b:67:2d:ac:f1:c2:
    da:17:78:c4:0d:df:76:e0:11:eb:c5:af:c5:4f:07:
    87:63:cf:c1:9f:a1:c2:3b:5b:12:85:61:82:3e:4f:
    ef:c8:ae:5d:77
prime2:
    00:ef:3d:47:1e:30:a3:91:8a:fc:52:d7:e9:39:70:
    9c:d7:60:b1:aa:3e:7b:7e:7b:7b:7d:c3:c1:33:fd:
    ff:89:68:e1:78:61:94:0f:51:15:22:ab:ce:b3:d8:
    57:e7:f0:eb:d1:06:2b:38:dc:0a:9d:16:2f:b1:d7:
    d1:67:fb:9d:a3
exponent1:
    78:56:9b:bf:de:e3:4a:10:17:f7:b1:7f:44:32:eb:
    ec:e7:6e:87:af:51:42:62:22:9e:7c:81:0e:9e:70:
    8f:18:47:12:af:ec:9d:fc:a7:72:c1:f7:8b:6c:ac:
    5b:f2:e5:26:2e:31:72:2f:9a:79:15:d7:86:c1:3d:
    c1:fc:34:af
exponent2:
```

On peut aussi afficher la clé publique avec `openssl rsa -in public_f.pem -pubin -text -noout` :

```
bluefir  TP1  main  openssl rsa -in public_f.pem -pubin -text -noout
Public-Key: (1024 bit)
Modulus:
    00:e2:8c:21:23:11:5c:94:bd:9a:8f:6b:39:67:81:
    68:16:72:5a:eb:47:bd:96:43:5a:a6:2f:3e:a5:d6:
    13:77:5a:bd:af:f8:b3:45:6c:71:22:a1:a0:6a:42:
    ed:2d:f8:b2:58:4b:ce:ce:ea:84:de:1b:40:7f:a3:
    02:bd:e2:57:17:77:98:52:05:1c:87:8b:c7:d9:2b:
    fc:e1:52:ca:ce:73:06:0d:ce:e5:25:cb:9a:b6:ff:
    f2:c7:a0:75:8c:5a:59:ab:bf:4a:ab:f8:cd:36:7f:
    cf:b7:c4:e8:a2:af:7e:58:51:42:23:10:68:30:ea:
    76:bd:ab:1d:77:3b:04:7d:c5
Exponent: 65537 (0x10001)
```

11. Les nombres premiers sont de taille 512 bits car la taille de la clé RSA est de 1024 bits et que la clé RSA est le produit de deux nombres premiers de taille égale.
    On chiffre la clé avec DES3 en utilisant la commande `openssl rsa -in private_f.pem -des3 -out private_f_des3.pem`:

```
bluefir  TP1  main  openssl rsa -in private_f.pem -des3 -out private_f_des3.pem
writing RSA key
Enter pass phrase:
Verifying - Enter pass phrase:
bluefir  TP1  main  cat private_f_des3.pem
-----BEGIN ENCRYPTED PRIVATE KEY-----
MIIC1DBOBgkqhkiG9w0BBQ0wQTApBgkqhkiG9w0BBQwwHAQI1DQIoEjza3wCAggA
MAwGCCqGSIb3DQIJBQAwFAYIKoZIhvcNAwcECIdJONU9JcjUBIICgLlWML+E4G53
BzRve0H291bqUXLqUpQmocg5Z0mJ8vaHZx+cgq2zjt+IH7F85ORJCzq44FlLruz9
YanopMvtNqQpZku7UMNth+qiN2YzkR8oZzt01Q1Uhg7ONlHVkjJdYq4qe8yR/vcJ
/KFcwO+vzvxSQRJyR4WwzPPrBACQBX8SSa7kmD93MiOAzlPtJAI0RCW/gNpeosmE
I9UK+QIdXQHP+mAnAxMf5hVzr8SrJdsyngquaAJUUSg1QmeDNosItpbe+FwG9TBy
1V06ca4ykNfnhoWh1cKoWy9tzvYk/qfTgMEGBDHLEI2LDEqYGNQKaaiyAPXkEvRd
n2CsINKEazU49FRl+QFpV+UxINlAy4h2S43vPsxr5X07slSibM35PR7s2hj5/6fP
1N7+F23hsSLlPTtSsDjnoxGdv4t8RFNMavtwpcjOh27ydYK5k8mFt+QVrFtTWSXC
tcvMayxc0prYx+TjYmNwKNC8KiLDnjV4D79HO0S1rizDbd0U12GX6JT5QZ0Y5I/K
z4kTg2509RJ+0AZSZvkyvne1F2H5FyhOo+2763P8ils0jfbQK79qQBuWlGkTG7H/
6Sc1Dnr5pzPW8TjY9V3eN8rlPH0C0a2Mf3DdLtXxPA5jv2P85rAbqP9q+8rS3U2D
nAS0ScITyNwgyHfC0wttO8naF0SoHYJxBnmVYQyT3BtxDJ+x7NDlJyT6PqCWhcDZ
syIWPvSVu7p8vat7Y3Uz2+36kw6wWPO6d9JYxvmgX72wWUNGy41xFz/5DykyCjTG
7guv646fJc+GZ6KuW+E9u3RLP3snY9Tp5VzxxzIMrpuMbZI08SoWadWHSMuenWj7
Tgij0oUmhw4=
-----END ENCRYPTED PRIVATE KEY-----
```

    On a déjà séparé la clé publique dans le fichier `public_f.pem` à la question précédente.

    Le but de chiffrer la clé privée avec DES3 est de protéger la clé privée contre les accès non autorisés. Si la clé privée est stockée en clair, toute personne ayant accès au fichier peut l'utiliser pour déchiffrer les messages ou pour signer des documents. En chiffrant la clé privée avec un mot de passe, on ajoute une couche de sécurité supplémentaire, car même si quelqu'un accède au fichier, il ne pourra pas utiliser la clé privée sans connaître le mot de passe.

Partie 4 : Chiffrement asymétrique

On va chiffrer 2 fichiers avec la clé publique générée à la question précédente avec `openssl rsautl -pubin -inkey public_f.pem -encrypt -oaep -out secret1` et `openssl rsautl -pubin -inkey public_f.pem -encrypt -oaep -out secret2`:

```
bluefir  TP1  main  echo "secret message" | openssl rsautl -pubin -inkey public_f.pem -encrypt -oaep -out secret1
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
```

```
bluefir  TP1  main  echo "another secret message" | openssl rsautl -pubin -inkey public_f.pem -encrypt -oaep -out secret2
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
```

12. La taille du fichier chiffré est de 128 octets (1024 bits) car la taille du bloc chiffré avec RSA est égale à la taille de la clé RSA. Par contre si le message à chiffrer est plus grand que la taille de la clé, `rsautl` ne pourra pas le chiffrer.

```
bluefir  TP1  main  ls -l src/secret1
-rw-r--r-- 1 bluefir bluefir 128 Jan 20 22:10 src/secret1
```

13. Les fichiers sont différents car le chiffrement RSA avec OAEP utilise un padding aléatoire, ce qui signifie que même si le même message est chiffré plusieurs fois avec la même clé publique, les résultats seront différents à chaque fois. Cela améliore la sécurité du chiffrement en empêchant les attaques par analyse de fréquence ou par comparaison de messages chiffrés.
On vérifie que les fichiers chiffrés sont différents avec `cmp secret1 secret2`:

```
bluefir  TP1  main  cmp secret1 secret2
secret1 secret2 differ: byte 1, line 1
```

De plus, en observant leur contenu en hexadécimal, on peut voir que les octets sont différents et qu'il n'y a pas de motifs alors que le message d'origine est similaire:

```
TP1 > secret2
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   Decoded Text
00 F4 91 BE A4 47 30 06 CD B0 DC 51 67 02 E5 2C   . . . . . G 0 . . . . Q g . . ,
2A 4A 31 81 AC A5 BB E9 E9 6A AF FE 66 FB 14 0A   * J 1 . . . . . j . . f . . .
88 E4 E3 53 9D CA 4A 97 3E 6B 3B 9A F6 72 25 0C   . . . S . . J . > k ; . . r % .
99 E7 AB 24 03 6C 33 D2 02 DB FC 9C 89 AA 28 7F   . . . $ . l 3 . . . . . . . ( .
8F 6B BB 76 3C AF C0 59 BC E4 D0 10 83 09 AE 4E   . k . v < . . Y . . . . . . . N
01 E9 1D 46 CA 12 44 08 97 7F 30 56 44 0F F3 E7   . . . F . . D . . . 0 V D . . .
01 1F B4 15 0F A0 EB F1 BD E4 AC C5 9C 55 91 B2   . . . . . U . .
67 33 DE EC A9 1A CC 03 20 6D C4 59 A1 C8 89 34   g 3 . . . . . m . Y . . . 4
+                                                 +
```

```
TP1 > secret1
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F   Decoded Text
29 33 69 76 0E AB 49 B7 C7 F8 33 6E C0 91 C5 28   ) 3 i v . . I . . . 3 n . . . (
40 7E 1D CF 05 1C 2B 2F 37 FC B7 D7 F6 C5 19 94   @ ~ . . . . . + / 7 . . . . . .
FF B4 28 2B 46 59 2D B9 C1 D0 DA 79 0E B1 3D 24   . . ( + F Y - . . . . y . . = $
A4 D9 D6 DA 7B 6D 3A 3E D2 80 B7 91 42 AA 3A 45   . . . . { m : > . . . . B . : E
3C AF 50 DE 18 18 83 1D 05 5A CE B6 7E DA D4 22   < . P . . . . . Z . . ~ . . "
27 82 C6 0B 43 BF 41 CB 48 0B C9 7F 35 8B 67 7A   ' . . . C . A . H . . . 5 . g z
BD C4 F0 8C A9 A7 E7 C4 CE 04 77 16 DE 68 A0 BA   . . . . . w . . h . .
E4 7C 13 BF 7F 91 2F A9 69 FC 9F 57 DF E0 DE A6   . | . . . . / . i . . W . . . .
+                                                 +
```

On vérifie le déchiffrement avec `openssl rsautl -inkey private_f.pem -decrypt -oaep -in secret1`:

```
bluefir  TP1  main  openssl rsautl -inkey private_f.pem -decrypt -oaep -in secret1
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
secret message
```

Et pour le second fichier :

```
bluefir  TP1  main  openssl rsautl -inkey private_f.pem -decrypt -oaep -in secret2
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
another secret message
```

14. La commande demande un mot de passe car la clé privée a été chiffrée avec un mot de passe à la question 11. Pour pouvoir utiliser la clé privée pour déchiffrer les messages, il faut d'abord la déchiffrer en fournissant le mot de passe correct. Cela garantit que seule une personne autorisée ayant connaissance du mot de passe peut accéder à la clé privée et l'utiliser pour déchiffrer les messages.

Partie 5 : Signature

On signe le fichier /etc/passwd avec la clé privée générée précédemment avec la commande `openssl dgst -sha256 -out passwd.sig -sign private_f.pem /etc/passwd` :

```
bluefir  TP1  main  openssl dgst -sha256 -out passwd.sig -sign private_f.pem /etc/passwd
```

On peut vérifier la signature avec la clé publique avec la commande `openssl dgst -sha256 -signature passwd.sig -verify public_f.pem /etc/passwd` :

```
bluefir  TP1  main  openssl dgst -sha256 -signature passwd.sig -verify public_f.pem /etc/passwd
Verified OK
```

15. La première commande demande un mot de passe car elle utilise la clé privée qui a été chiffrée avec un mot de passe à la question 11. Pour pouvoir utiliser la clé privée pour signer le fichier, il faut d'abord la déchiffrer en fournissant le mot de passe correct. Cela garantit que seule une personne autorisée ayant connaissance du mot de passe peut accéder à la clé privée et l'utiliser pour signer des fichiers. La deuxième commande ne demande pas de mot de passe car elle utilise la clé publique, qui n'est pas chiffrée et peut être utilisée librement pour vérifier les signatures.

16. Si on modifie le fichier /etc/passwd et qu'on vérifie à nouveau la signature, on obtient le message d'erreur suivant :

```
bluefir  TP1  main  sudo vim /etc/passwd
[sudo] password for bluefir:
bluefir  TP1  main  openssl dgst -sha256 -signature passwd.sig -verify public_f.pem /etc/passwd
Verification failure
40376548D7750000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:430:
40376548D7750000:error:1C880004:Provider routines:rsa_verify:RSA lib:../providers/implementations/signature/rsa_sig.c:774:
```

Le message indique que la signature n'est pas valide pour le fichier modifié. En effet, la signature est calculée à partir du contenu du fichier original, donc si le fichier est modifié, la signature ne correspond plus au nouveau contenu. Cela montre l'intégrité du fichier, car toute modification du fichier entraîne une invalidation de la signature.

17. Non, on ne peut pas savoir ce qui a été modifié en utilisant uniquement la signature. La signature permet seulement de vérifier si le fichier a été modifié ou non, mais elle ne fournit pas d'informations sur les modifications spécifiques apportées au fichier. Pour savoir ce qui a été modifié, il faudrait comparer le fichier original avec le fichier modifié en utilisant des outils de comparaison de fichiers, comme `diff` sous Linux.

Partie 6 : Echange de fichiers chiffrés et signés

18. Supposons que le fichier à transférer est `monfichier.txt` depuis `source/` vers `destination/`. Voici les commandes utilisées pour préparer le fichier à envoyer et pour le recevoir, en créant une paire de clés propre au destinataire (séparation claire des identités) :

    1. Préparation des répertoires et génération (si nécessaire) des clés du destinataire :

    ```
    mkdir -p source destination
    # Générer une paire de clés pour le destinataire (destination)
    openssl genrsa -out destination/private_dest.pem 1024
    openssl rsa -in destination/private_dest.pem -pubout -out
    destination/public_dest.pem

    # Copier la clé publique du destinataire dans le répertoire de
    l'expéditeur
    cp destination/public_dest.pem source/

    # Copier la clé publique de l'expéditeur (générée précédemment :
    public_f.pem)
    # dans le répertoire du destinataire pour vérification des
    signatures
    cp public_f.pem destination/
    ```

    Ces commandes créent les répertoires `source` et `destination`, génèrent une paire de clés propre au destinataire (`private_dest.pem` / `public_dest.pem`), puis distribuent les clés publiques nécessaires : la clé publique du destinataire est copiée dans `source/` pour permettre à l'expéditeur de chiffrer pour le destinataire, et la clé publique de l'expéditeur (`public_f.pem`) est copiée dans `destination/` pour permettre la vérification des signatures.

    2. L'expéditeur chiffre le fichier avec la clé publique du destinataire (précédemment copiée dans `source/`) :

    ```
    openssl rsautl -encrypt -oaep -pubin -inkey
    source/public_dest.pem -in source/monfichier.txt -out
    source/monfichier.txt.enc
    ```

    Cette commande chiffre `monfichier.txt` avec la clé publique du destinataire en utilisant le padding OAEP, et produit `monfichier.txt.enc`.

    3. L'expéditeur signe le fichier original avec sa clé privée (garantit son authenticité et son intégrité) :

```
openssl dgst -sha256 -sign source/private_f.pem -out
source/monfichier.txt.sig source/monfichier.txt
```

Cette commande calcule le haché SHA-256 du fichier et signe ce haché avec la clé privée de l'expéditeur, écrivant la signature dans `monfichier.txt.sig`.

4. On transfère le fichier chiffré et la signature :

```
cp source/monfichier.txt.enc destination/
cp source/monfichier.txt.sig destination/
```

Ces commandes transfèrent respectivement le fichier chiffré et la signature vers le répertoire du destinataire.

5. Le destinataire déchiffre le fichier avec sa clé privée :

```
openssl rsautl -decrypt -oaep -inkey destination/private_dest.pem
-in destination/monfichier.txt.enc -out
destination/monfichier.txt
```

Cette commande utilise la clé privée du destinataire (`private_dest.pem`) pour déchiffrer `monfichier.txt.enc` et restaurer le fichier original.

19. Les commandes utilisées pour signer le fichier, envoyer la signature puis vérifier cette signature sont les suivantes :

1. L'expéditeur signe le fichier avec sa clé privée :

```
openssl dgst -sha256 -sign source/private_f.pem -out
source/monfichier.txt.sig source/monfichier.txt
```

Cette commande calcule le digest SHA-256 de `monfichier.txt` et signe ce digest avec la clé privée `source/private_f.pem`, produisant la signature `monfichier.txt.sig`.

2. On transfère la signature et la clé publique de l'expéditeur vers le destinataire :

```
cp source/monfichier.txt.sig destination/
cp source/public_f.pem destination/
```

Ces commandes transfèrent la signature et la clé publique de l'expéditeur vers le destinataire afin que celui-ci puisse vérifier la signature.

3. Le destinataire vérifie la signature avec la clé publique de l'expéditeur :

```
openssl dgst -sha256 -verify destination/public_f.pem -signature
destination/monfichier.txt.sig destination/monfichier.txt
```

La signature est valide si la commande retourne "Verified OK". Cela signifie que le fichier n'a pas été modifié depuis qu'il a été signé par l'expéditeur, garantissant ainsi son intégrité et son authenticité.

Cette commande vérifie la signature en recalculant le haché SHA-256 du fichier reçu et en utilisant la clé publique fournie pour valider que la signature provient bien de la clé privée associée.